LAB ASSIGNMENT 15.2

Lab 15 – Backend API Development: Creating RESTful Services with AI

Name: U.V.Manohar Reddy

Rollno:2503a52397

Batch-14

Task 1: Setup FastAPI Backend

2. Prompt :

Create a basic FastAPI app with a root GET endpoint that returns a welcome message.

3. Code

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    """Return a welcome message."""
    return {"message": "Welcome to AI-powered FastAPI"}
```

4. Output



5. Justification

FastAPI creates the backend application. The GET endpoint at / returns a welcome message and confirms that the server is running.

Task 2: API – Create and Read Items

2. Prompt (Small)

Add GET and POST endpoints to retrieve and create items using a Python list.

3. Code

```
items = []

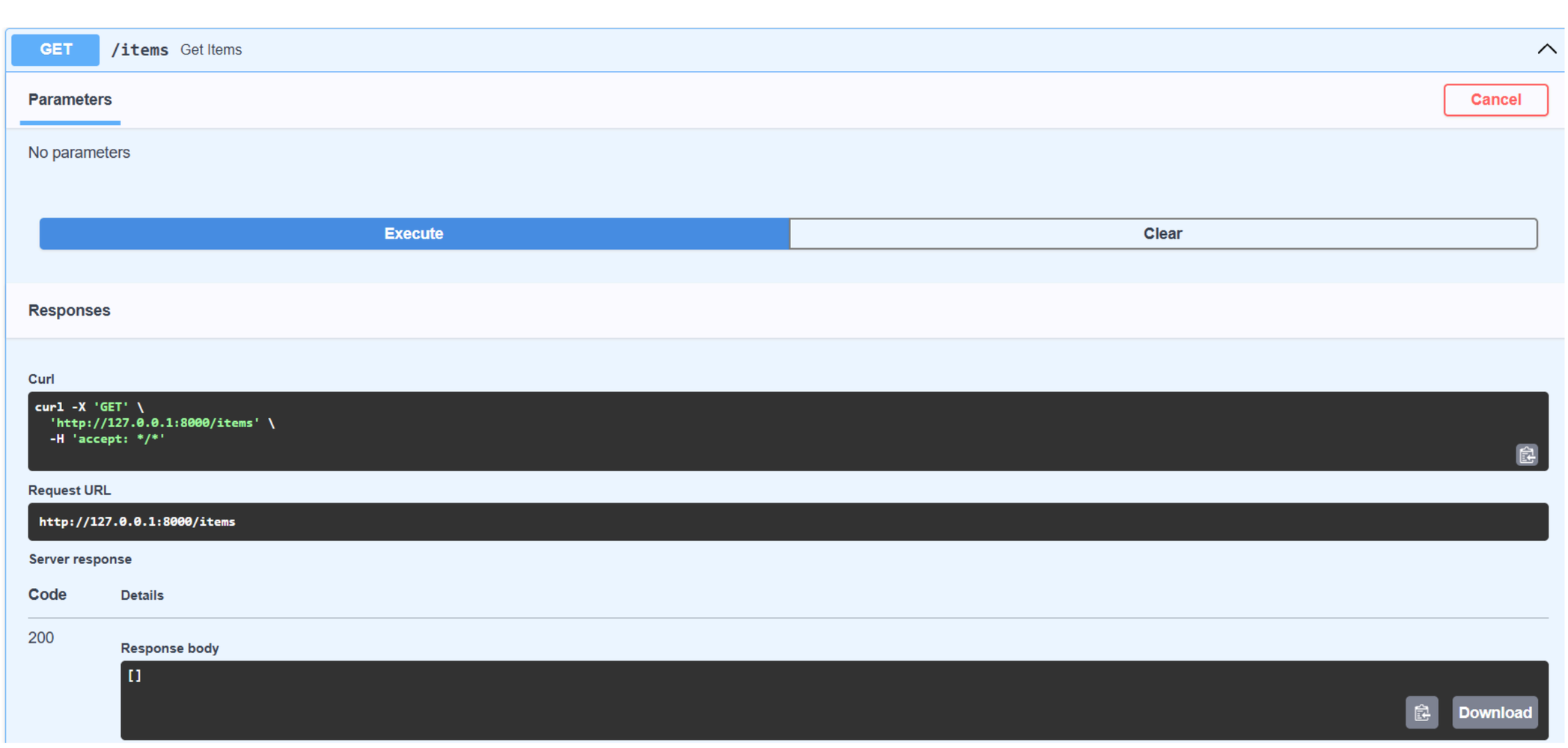
@app.get("/items")
```

```
def get_items():
    """Return all stored items."""
    return items

@app.post("/items")
def add_item(item: dict):
    """Add an item to the list."""
    items.append(item)
    return {"message": "Item created", "item": item}
```

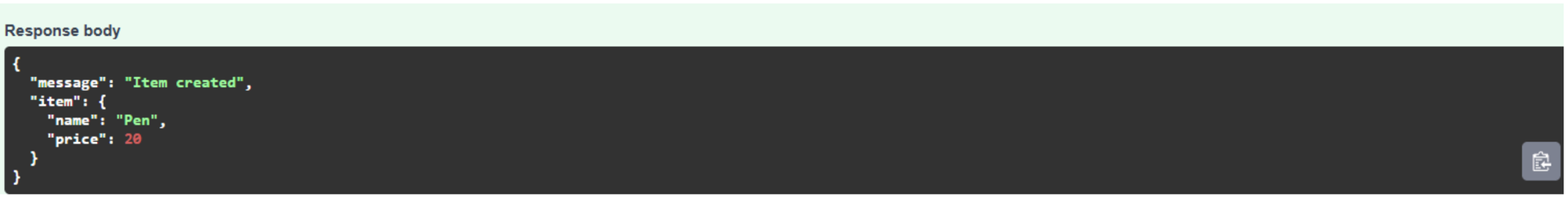
4. Output

GET /items :



POST /items :

Response body



```
{
  "message": "Item created",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```

5. Justification

The GET endpoint displays the items in the list. The POST endpoint accepts an item as JSON, stores it in the list, and returns a confirmation message.

Task 3: Update Existing Item

2. Prompt (Small)

Create a PUT endpoint that updates an item by its zero-based index and reports when the item is not found.

3. Code

```
@app.put("/items/{index}")
def update_item(index: int, item: dict):
    """Update an item at the given index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    items[index] = item
    return {"message": "Item updated", "item": item}
```

4. Output

PUT /items:

Name | Description

index * required
integer
(path)

0

Request body required — application/json

Edit Value | Schema

```
{
  "name": "Pencil",
  "price": 10
}
```

Request URL

```
http://127.0.0.1:8000/items/0
```

Server response

Code	Details
200	Response body

```
{
  "message": "Item updated",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```

Response headers

```
content-length: 62
content-type: application/json
date: Wed,28 Oct 2026 06:04:16 GMT
server: uvicorn
```

Responses

5. Justification

The PUT endpoint replaces the item at the specified index. It returns a 404 error if the index does not refer to an existing item.

Task 4: Delete Item

2. Prompt (Small)

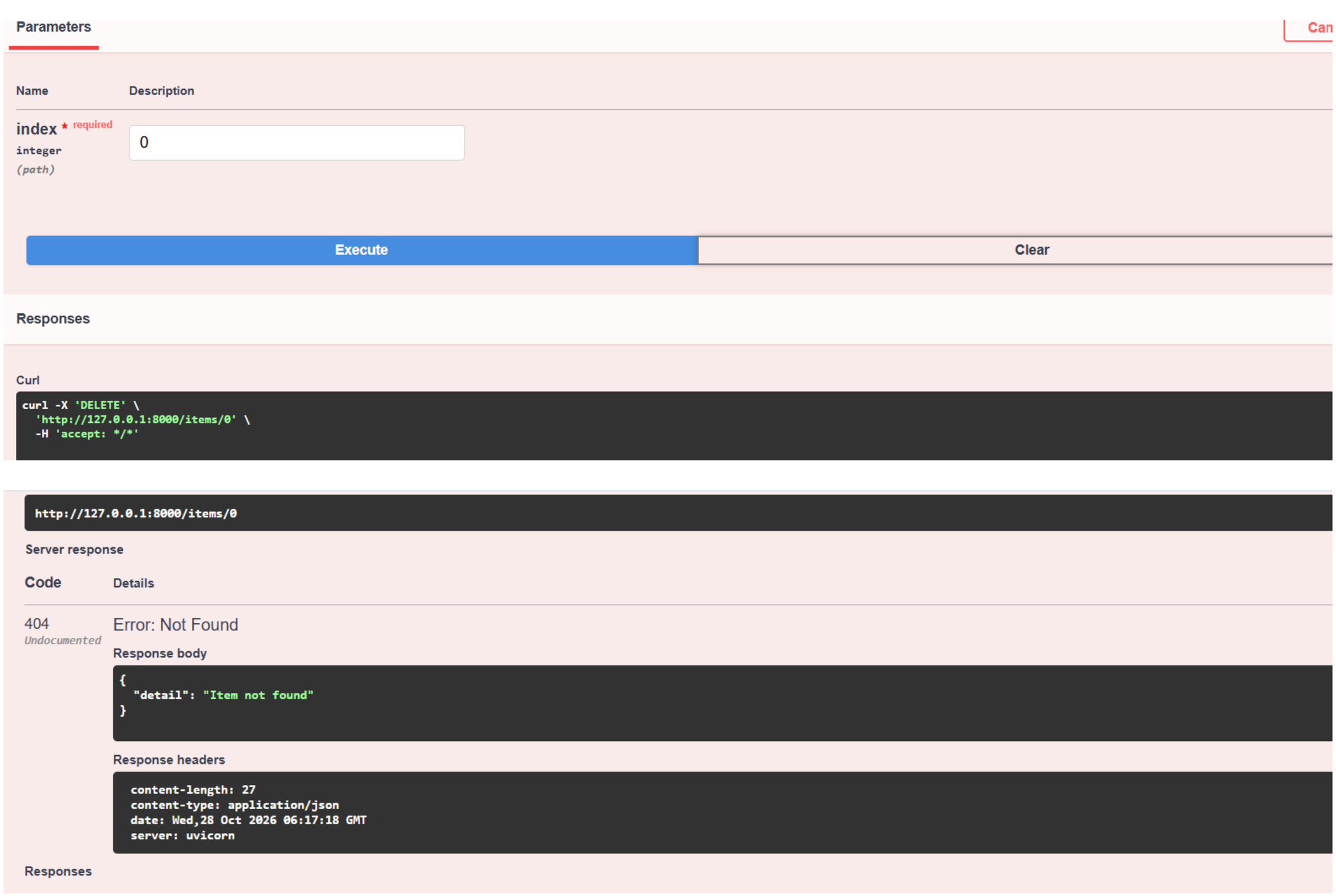
Create a DELETE endpoint that removes an item by its index and returns the deleted item.

3. Code

```
@app.delete("/items/{index}")
def delete_item(index: int):
    """Delete an item at the given index."""
    if index < 0 or index >= len(items):
        raise HTTPException(status_code=404, detail="Item not found")
    removed = items.pop(index)
    return {"message": "Item removed", "item": removed}
```

4. Output

DELETE /items/0:



5. Justification

The DELETE endpoint removes the selected item from the list and returns the removed item. Invalid indexes produce a 404 error.

Task 5: Add Auto-Generated Documentation

2. Prompt (Small)

Add comments, docstrings, endpoint summaries, and Swagger documentation to the FastAPI endpoints.

3. Code

```
from fastapi import FastAPI, HTTPException

app = FastAPI(
    title="Items CRUD API",
    description="API for creating, reading, updating, and deleting items",
    version="1.0.0"
)

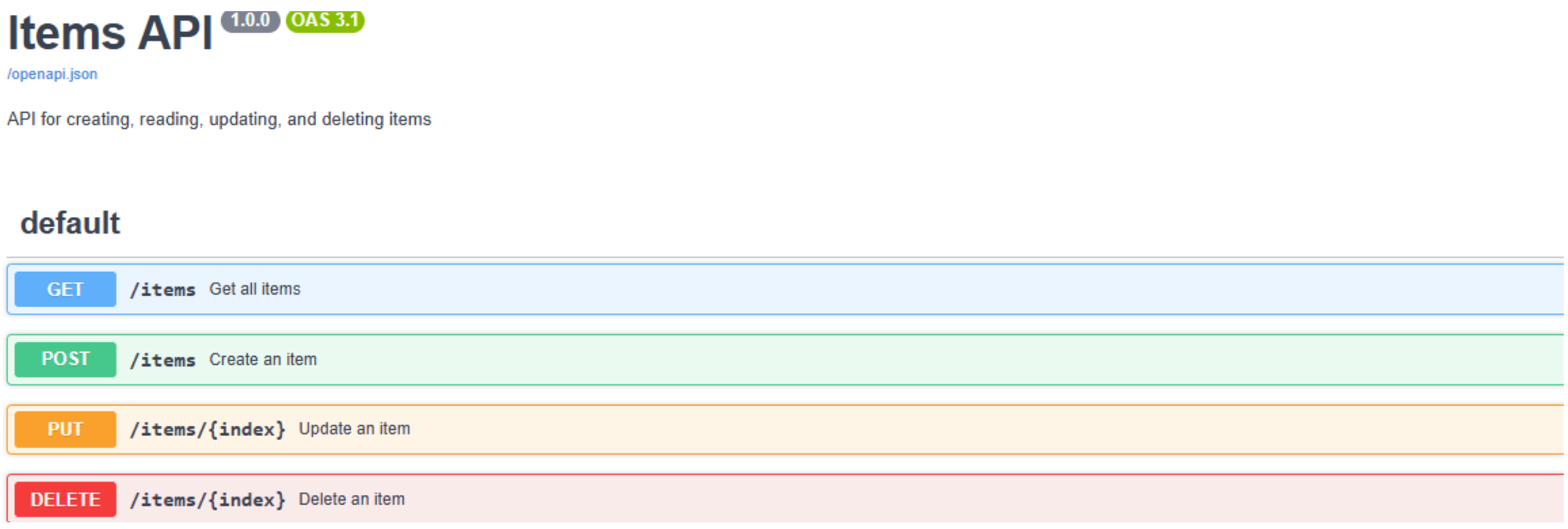
@app.get("/items", summary="Get all items")
def get_items():
    """Return all items stored in memory."""
    return items

@app.post("/items", summary="Create an item")
def add_item(item: dict):
    """Create and return a new item."""
    items.append(item)
    return {"message": "Item created", "item": item}
```

Add summaries and docstrings to the PUT and DELETE endpoints in the same way.

4. Output

Run the server and open:



Swagger UI displays the available endpoints, HTTP methods, descriptions, and request body fields.

5. Justification

FastAPI reads endpoint definitions, type hints, summaries, and docstrings to generate interactive Swagger documentation automatically. The /docs page helps users test the API without needing a separate API client.

Complete Program Note

For a single runnable program, combine the endpoint code from Tasks 1– 4 in one main.py file. Include the imports at the top: from fastapi import FastAPI, HTTPException. Define app = FastAPI() and items = [] only once. Task 5 documentation metadata can be added to that same FastAPI instance.

Run the completed program with: uvicorn main:app --reload

Note: The item list is temporary storage. Its contents are cleared when the server restarts.